

กลุ่มที่ 4 Expression Processor : Infix และ Postfix

สมาชิก

นายกษิตศ อุ่่นอำไพ 68122250028

นางสาวน้ำผึ้ง เอียดวงศ์ 68122250031

นายชลสิทธิ์ จิตรณรงค์ 68122250092

นายณัฐชนันท์ จินดานิล 68122250102

นายพฤทธชาติ ดำนิล 68122250104

ที่มาและความสำคัญ

นิพจน์ทางคณิตศาสตร์สามารถเขียนได้หลายรูปแบบ โดยรูปแบบที่มนุษย์คุ้นเคยคือ Infix Expression เช่น $3 + 4 * 2$ ซึ่ง Operator จะอยู่ระหว่าง Operand อย่างไรก็ตาม การประมวลผลนิพจน์จำเป็นต้องพิจารณาลำดับความสำคัญของ Operator และการใช้วงเล็บให้ถูกต้อง

โครงการนี้จึงพัฒนาโปรแกรม Expression Processor สำหรับรับและประมวลผลนิพจน์ทางคณิตศาสตร์ โดยเปรียบเทียบ Algorithm จำนวน 2 วิธี ได้แก่

- Algorithm A: แปลง Infix เป็น Postfix ก่อน แล้วจึงประเมินผลลัพธ์
- Algorithm B: ประเมิน Infix โดยตรงโดยใช้ Operand Stack และ Operator Stack

นอกจากการคำนวณผลลัพธ์แล้ว ระบบยังต้องสามารถตรวจสอบ Input ที่ไม่ถูกต้อง เช่น วงเล็บไม่สมดุล Operator เรียงผิดรูปแบบ และการหารด้วยศูนย์ รวมถึงเปรียบเทียบประสิทธิภาพของ Algorithm ทั้งสองวิธี

วัตถุประสงค์

1. วิเคราะห์ Input, Output และข้อจำกัดของปัญหา
2. ออกแบบ Algorithm สำหรับปัญหาดังกล่าวอย่างน้อย 2 วิธี
3. เลือกใช้ Stack และโครงสร้างข้อมูลอื่นอย่างเหมาะสม
4. เขียน Pseudocode และอธิบายขั้นตอนการทำงานของ Algorithm
5. อธิบายและพิสูจน์ความถูกต้องของ Algorithm
6. วิเคราะห์ Time Complexity และ Space Complexity
7. พัฒนา Algorithm เป็นโปรแกรมภาษา Java
8. ออกแบบ Test Cases และทดสอบกรณีปกติและกรณีขอบเขต
9. ทดลองเปรียบเทียบประสิทธิภาพของ Algorithm A และ Algorithm B
10. สรุปผลและเลือก Algorithm ที่เหมาะสมสำหรับปัญหา

Case Story

ระบบ Expression Processor เป็นโปรแกรมสำหรับรับนิพจน์ทางคณิตศาสตร์ในรูปแบบ Infix แล้วตรวจสอบความถูกต้องและคำนวณผลลัพธ์

Input , Output และ Constraints

Input

Input คือ Mathematical Expression ในรูปแบบ Infix เช่น

$$3 + 4 * 2$$

$$(3 + 4) * 2$$

$$10 / (5 - 5)$$

Output

ระบบต้องแสดงผลของนิพจน์ที่ถูกต้อง เช่น

Input: $3 + 4 * 2$

Result: 11

หาก Input ไม่ถูกต้อง ระบบต้องแจ้ง Error ที่เหมาะสม

นอกจากนี้ในการทดลอง Performance ระบบต้องเก็บข้อมูล เช่น

1. จำนวน Operation
2. Execution Time
3. ผลลัพธ์ของ Algorithm A
4. ผลลัพธ์ของ Algorithm B

Constraints

ระบบรองรับ:

1. Operand เป็นจำนวนเต็ม
2. Operator +
3. Operator -
4. Operator *
5. Operator /
6. วงเล็บ (และ)
7. จำนวนหลายหลัก
8. ช่องว่างระหว่าง Token

ระบบไม่จำเป็นต้องรองรับ:

1. Decimal
2. Unary Minus
3. ตัวแปร
4. Function
5. []
6. {}

สมมติฐาน

กำหนดข้อตกลงในการทำงานดังนี้

1. Input ถูกส่งเข้ามาในรูปแบบ String
2. Operand เป็นจำนวนเต็ม
3. Operator ที่ใช้คือ +, -, *, /
4. วงเล็บที่ใช้คือ (และ)
5. ระบบต้องตรวจสอบความถูกต้องก่อนประมวลผล
6. การหารด้วยศูนย์ถือเป็น Error

Algorithm A

Algorithm A: Infix to Postfix Evaluation

แนวคิดหลัก

Algorithm A แบ่งการทำงานออกเป็น 2 ขั้นตอนหลัก

ขั้นตอนที่ 1

Infix $\rightarrow$ Postfix

ขั้นตอนที่ 2

Postfix $\rightarrow$ Result

ตัวอย่าง:

Infix:

$3 + 4 * 2$

แปลงเป็น

Postfix:

$3\ 4\ 2\ *\ +$

จากนั้นประเมิน Postfix เพื่อให้ได้

Result = 11

Input

Infix Expression

Output

Result

หรือ Error หาก Input ไม่ถูกต้อง

ขั้นตอนการทำงาน

1. รับ Infix Expression
2. Tokenize Expression
3. ตรวจสอบความถูกต้องของ Expression
4. อ่าน Token ทีละตัว
5. หากเป็น Operand ให้เพิ่มเข้า Output
6. หากเป็น Operator ให้พิจารณา Priority
7. หากเป็น (ให้ Push เข้า Operator Stack
8. หากเป็น) ให้ Pop Operator จนพบ (
9. เมื่ออ่าน Token ครบ ให้ Pop Operator ที่เหลือ

10. ได้ Postfix Expression

11. นำ Postfix ไปประเมินด้วย Operand Stack

12. แสดงผลลัพธ์

```
## Algorithm A
Input: Infix Expression
1. Tokenize expression
2. Validate expression
3. Create Operator Stack
4. Create Postfix Output
5. For each token:
    If token is Operand:
        Add token to Output
    Else if token is '(':
        Push token to Operator Stack
    Else if token is ')':
        Pop operators until '(' is found
        Remove '('
    Else if token is Operator:
        While top of stack has higher or equal priority:
            Pop operator to Output
        Push current operator
6. Pop remaining operators to Output
7. Create Operand Stack
8. For each token in Postfix:
    If token is Operand:
        Push token
    Else if token is Operator:
        Pop required operands
        Perform operation
        Push result
9. Return top of Operand Stack
```

ตัวอย่างการทำงาน

กำหนด

$3 + 4 * 2 / (1 - 5)$

ผลลัพธ์ Postfix คือ

3 4 2 * 1 5 - / +

จากนั้นประเมิน:

$$4 * 2 = 8$$

$$1 - 5 = -4$$

$$8 / -4 = -2$$

$$3 + (-2) = 1$$

ดังนั้น

Result = 1

ข้อดีของ Algorithm A

1. แยกขั้นตอน Conversion และ Evaluation อย่างชัดเจน
2. Postfix ไม่มีปัญหาเรื่อง Operator Precedence ในขั้นตอน Evaluation
3. สามารถตรวจสอบและอธิบายการทำงานของ Stack ได้ง่าย
4. เหมาะสำหรับการศึกษาเรื่อง Stack และ Expression Processing

ข้อจำกัดของ Algorithm A

1. ต้องมีขั้นตอนแปลง Infix เป็น Postfix ก่อน
2. ต้องจัดการ Stack มากกว่าหนึ่งขั้นตอน
3. ต้องใช้พื้นที่สำหรับเก็บ Postfix Expression เพิ่มเติม

Algorithm B

Algorithm B: Direct Infix Evaluation

แนวคิดหลัก

Algorithm B ไม่แปลง Infix เป็น Postfix แต่ประเมิน Infix โดยตรง โดยใช้ Stack 2 ตัว ได้แก่

Operand Stack

Operator Stack

Input

Infix Expression

Output

Result

หรือ Error หาก Input ไม่ถูกต้อง

ขั้นตอนการทำงาน

1. รับ Infix Expression
2. Tokenize Expression
3. ตรวจสอบความถูกต้อง
4. สร้าง Operand Stack
5. สร้าง Operator Stack
6. อ่าน Token ทีละตัว
7. หากเป็น Operand ให้ Push ลง Operand Stack
8. หากเป็น Operator ให้ตรวจสอบ Priority
9. หากจำเป็นให้ประมวลผล Operator ก่อนหน้า
10. หากเป็น (ให้ Push เข้า Operator Stack
11. หากเป็น) ให้ประมวลผลจนพบ (
12. เมื่ออ่าน Token ครบ ให้ประมวลผล Operator ที่เหลือ
13. Result อยู่บน Operand Stack

```
# Algorithm B
```

```
Input: Infix Expression
```

1. Tokenize expression
2. Validate expression
3. Create Operand Stack
4. Create Operator Stack
5. For each token:


```
If token is Operand:
    Push token to Operand Stack
Else if token is '(':
    Push token to Operator Stack
Else if token is ')':
    While top operator is not '(':
        Apply operator
    Remove '('
Else if token is Operator:
    While top operator has higher or equal priority:
        Apply operator
    Push current operator
6. While Operator Stack is not empty:
    Apply operator
7. Return top of Operand Stack
```

ตัวอย่างการทำงาน

Input:

$3 + 4 * 2$

เริ่มต้น:

Operand Stack = empty

Operator Stack = empty

อ่าน 3

Operand Stack = [3]

Operator Stack = []

อ่าน +

Operand Stack = [3]

Operator Stack = [+]

อ่าน 4

Operand Stack = [3, 4]

Operator Stack = [+]

อ่าน *

เนื่องจาก * มี Priority สูงกว่า +

Operand Stack = [3, 4]

Operator Stack = [+ , *]

อ่าน 2

Operand Stack = [3, 4, 2]

Operator Stack = [+ , *]

เมื่อจบ Expression จะคำนวณ

$$4 * 2 = 8$$

$$3 + 8 = 11$$

ดังนั้น

$$\text{Result} = 11$$

ข้อดี

1. ไม่ต้องสร้าง Postfix Expression
2. ประเมิน Infix โดยตรง
3. สามารถประมวลผล Expression ได้ในขั้นตอนเดียว

ข้อจำกัด

1. ต้องจัดการ Operand Stack และ Operator Stack พร้อมกัน
2. Logic ในการจัดการ Priority และวงเล็บมีความซับซ้อน
3. การตรวจสอบ Error ต้องทำอย่างระมัดระวัง

Step-by-step

ตัวอย่าง Step-by-step เปรียบเทียบ

ใช้ Expression เดียวกัน:

$$3 + 4 * 2 / (1 - 5)$$

Algorithm A

Token	Operator Stack	Output
3		3
+	+	3
4	+	34
*	+*	34
2	+*	342
/	+ /	342*
(	+ / (	342*
1	+ / (	342*1
-	+ / (-	342*1
5	+ / (-	342*15
)	+ /	5-

สุดท้าย:

Postfix:

3 4 2 * 1 5 - / +

และ

Result = 1

Algorithm B

Token	Operator Stack	Output
3	3	

+	3	+
4	3,4	+
*	3,4	+, *
2	3,4,2	+, *
/	หลังประมวลผล *	+, /
(	...	+, /, (
1	...	+, /, (
-	...	+, /, (, -
5	...	+, /, (, -
)	...	+, /

เมื่อจบจะได้

Result = 1

Correctness Argument

Precondition

ก่อนเริ่ม Algorithm:

1. Input อยู่ในรูปแบบ Infix
2. Operand เป็นจำนวนเต็ม
3. Operator อยู่ในชุดที่ระบบรองรับ
4. Expression ผ่านการตรวจสอบรูปแบบเบื้องต้น

ในระหว่างการประมวลผล Token :

Token ที่ถูกประมวลผลแล้วจะถูกจัดเก็บใน Output หรือ Stack ในลักษณะที่ยังคงรักษาลำดับความสำคัญของ Operator และโครงสร้างของวงเล็บไว้

สำหรับ Algorithm A เมื่อประมวลผลแต่ละ Token แล้ว Output จะอยู่ในรูปแบบ Postfix ที่ถูกต้องสำหรับส่วนของ Expression ที่อ่านไปแล้ว

สำหรับ Algorithm B Operand Stack จะเก็บผลลัพธ์ย่อยของ Expression ที่ผ่านการประมวลผลแล้ว และ Operator Stack จะเก็บ Operator ที่ยังไม่ถูกประมวลผลตาม Priority

Postcondition

เมื่อ Algorithm ทำงานเสร็จ:

1. Algorithm A จะได้ Postfix ที่ถูกต้องและสามารถประเมินเป็นผลลัพธ์ที่ถูกต้อง
2. Algorithm B จะได้ผลลัพธ์ของ Infix Expression โดยตรง

ดังนั้นหาก Input ถูกต้อง ทั้งสอง Algorithm จะให้ผลลัพธ์เดียวกัน

Proof by Cases

กรณี Operand

Operand ถูกเก็บไว้ใน Output หรือ Operand Stack เพื่อรอการประมวลผล

กรณี Operator

ระบบตรวจสอบ Priority เพื่อให้ Operator ที่มี Priority สูงกว่าถูกประมวลผลก่อน

กรณี (

ถูก Push เข้า Stack เพื่อบังคับให้ Expression ภายในวงเล็บถูกประมวลผลก่อน

กรณี)

ระบบจะประมวลผล Operator จนพบ (แล้วนำ (ออกจาก Stack

Termination

Expression มีจำนวน Token จำกัด และแต่ละ Token จะถูกอ่านและประมวลผลตามลำดับ ดังนั้น Algorithm จะทำงานจนอ่าน Token ครบและสามารถสิ้นสุดการทำงานได้

Big-O Analysis

Balanced Parentheses

การตรวจสอบวงเล็บจะอ่าน Token แต่ละตัวเพียงครั้งเดียว

ดังนั้น

Time Complexity = $O(n)$

Space Complexity = $O(n)$

Algorithm A

ขั้นตอน Infix $\rightarrow$ Postfix อ่าน Token และ Push/Pop จาก Stack

แต่ละ Token จะถูก Push/Pop จำนวนจำกัด

ดังนั้นโดยรวม:

Time Complexity = $O(n)$

Space Complexity = $O(n)$

ขั้นตอน Postfix Evaluation ก็เป็น

$O(n)$

ดังนั้น Algorithm A โดยรวมเป็น

$O(n)$

Algorithm B

Algorithm B อ่าน Token แต่ละตัวและใช้ Operand Stack กับ Operator Stack

แต่ละ Operator ถูก Push และ Pop จำนวนจำกัด

ดังนั้น:

Time Complexity = $O(n)$

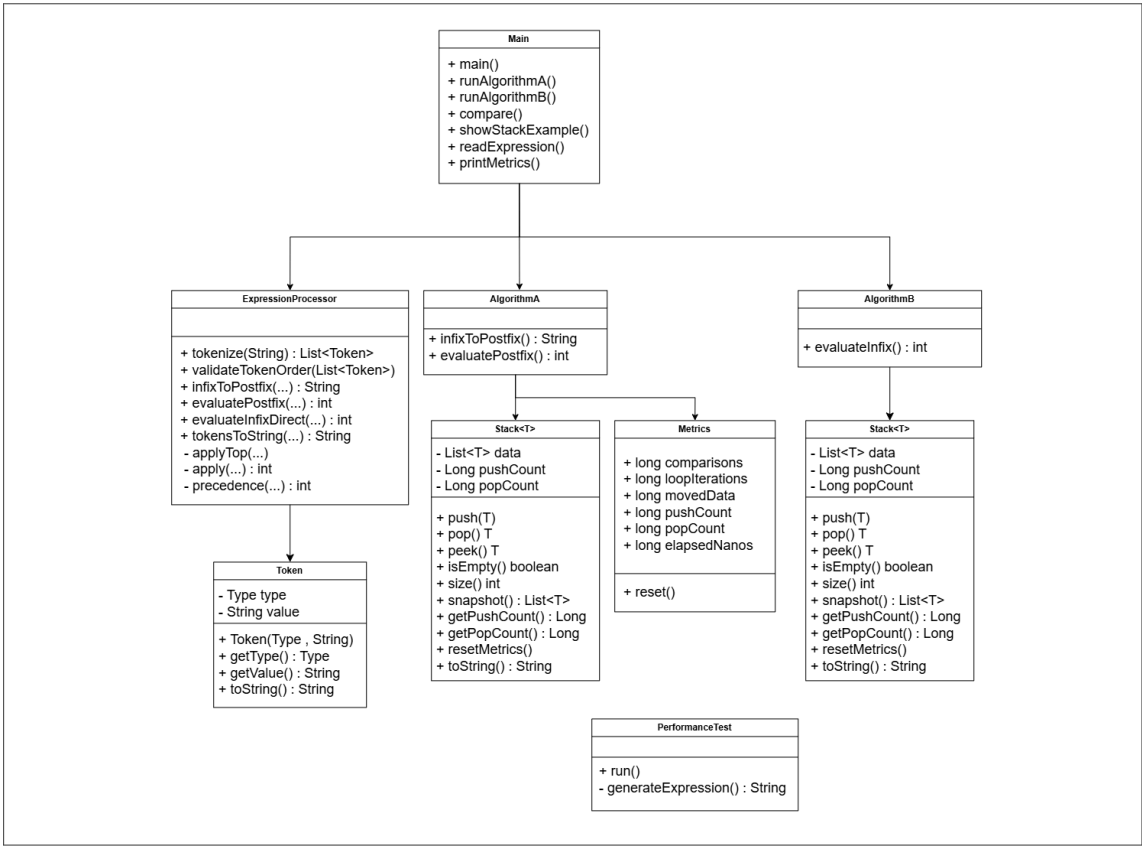
Space Complexity = $O(n)$

ตารางเปรียบเทียบ

	Algorithm A	Algorithm B
วิธี	Infix $\rightarrow$ Postfix $\rightarrow$ Evaluate	Direct Infix
Best Case	$O(n)$	$O(n)$
Average Case	$O(n)$	$O(n)$
Worst Case	$O(n)$	$O(n)$

Class Diagram และ Java Implementation

Class Diagram



Java Implementation

Class	หน้าที่
Main	เริ่มต้นโปรแกรมและจัดการเมนู
Token	แทนข้อมูล Token ของ Expression
Stack<T>	จัดเก็บ Operand และ Operator
ExpressionCommon	จัดเก็บ Operand และ Operator
AlgorithmA	แปลง Infix เป็น Postfix และประเมินผล
AlgorithmAB	ประเมิน Infix โดยตรง

Metrics	เก็บ Operation Count และข้อมูล Performance
PerformanceTest	ทำการทดลอง Performance

Test Cases

Test ID	Input Expression	Expected Result	Type	Algorithm A Result	Algorithm B Result	Status A	Status B
TC01	$3 + 4 * 2$	11	Valid	11	11	PASS	PASS
TC02	$(3 + 4) * 2$	14	Valid	14	14	PASS	PASS
TC03	$((8 + 2) * 5)$	50	Valid	50	50	PASS	PASS
TC04	$(3 + 4$	Error	Invalid	Error	Error	PASS	PASS
TC05	$3 + 4)$	Error	Invalid	Error	Error	PASS	PASS
TC06	$3 + * 4$	Error	Invalid	Error	Error	PASS	PASS
TC07	$10 / (5 - 5)$	Error	Invalid	Error	Error	PASS	PASS
TC08	$2 + 3 * 4 - 5$	9	Valid	9	9	PASS	PASS
TC09	$5 * 6 + 3$	33	Valid	33	33	PASS	PASS
TC10	$100 / 5 / 2$	10	Valid	10	33	PASS	PASS

หลักการตัดสิน Status

Expected Result = Actual Result → PASS

Expected Result ≠ Actual Result → FAIL

ดังนั้น Error Case ก็สามารถเป็น PASS ได้ หากโปรแกรมตรวจพบ Error ได้ถูกต้อง

Experimental Results

วัตถุประสงค์ของการทดลอง

เพื่อเปรียบเทียบประสิทธิภาพของ Algorithm A และ Algorithm B ในด้าน

1. Execution Time
2. Operation Count

3. Memory/Space

4. ความถูกต้องของผลลัพธ์

ขนาดข้อมูล

ใช้ข้อมูลอย่างน้อย 4 ขนาด:

100

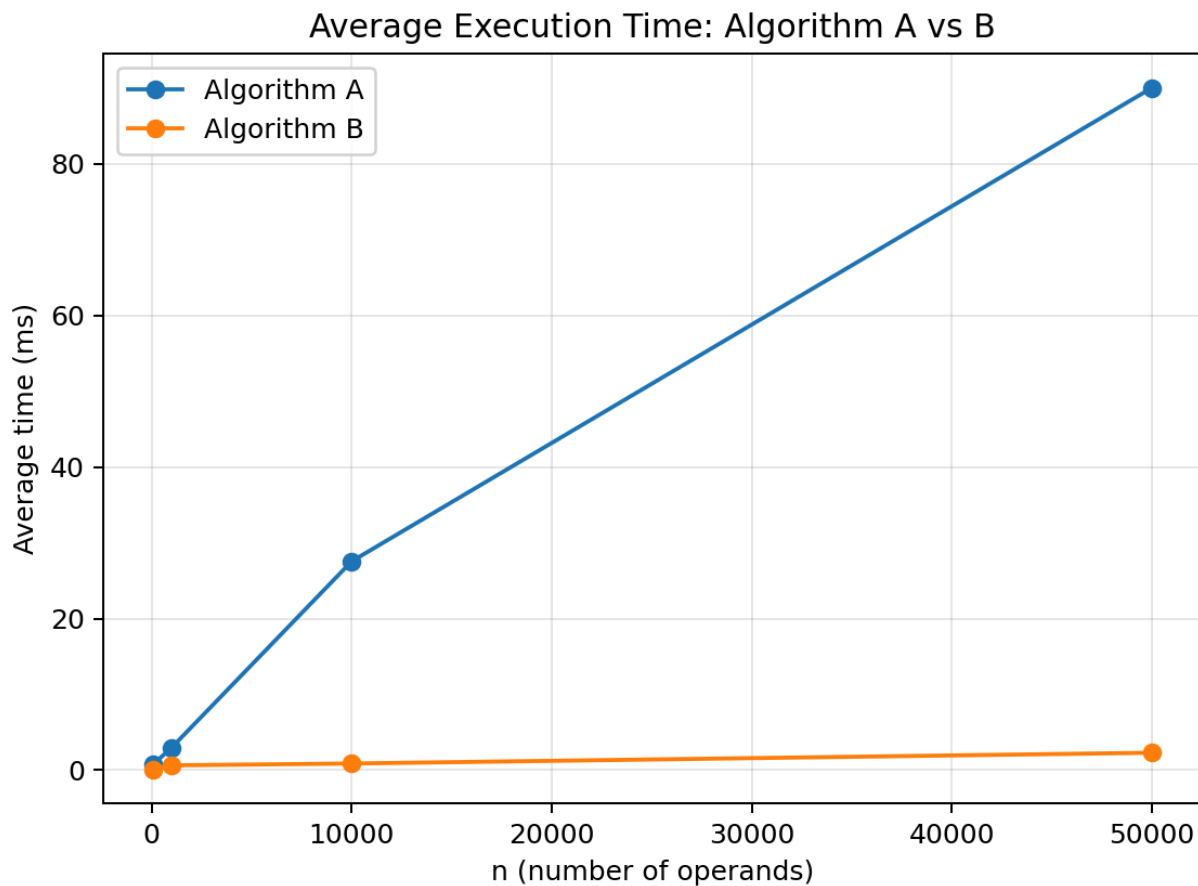
1,000

10,000

50,000

แต่ละขนาดทดลองอย่างน้อย 5 รอบ แล้วคำนวณค่าเฉลี่ย

n	algorithm	avg_time_ns	avg_operations	avg_comparator	avg_push	avg_pop	avg_loops
100	A	812809.4	1190	197	298	297	398
100	B	108253.2	991	197	298	297	199
1000	A	2969527	11990	1997	2998	2997	3998
1000	B	629617.6	9991	1997	2998	2997	1999
10000	A	27551009.8	119990	19997	29998	29997	39998
10000	B	869445.8	99991	19997	29998	29997	19999
50000	A	90035339.8	599990	99997	149998	149997	199998
50000	B	2291715.6	499991	99997	149998	149997	99999



สรุปและวิธีที่กลุ่มเลือก

สรุปผล

โครงการนี้ได้พัฒนา Expression Processor สำหรับประมวลผล Infix Expression ด้วย Algorithm 2 วิธี ได้แก่ Algorithm A ซึ่งแปลง Infix เป็น Postfix ก่อนประเมินผล และ Algorithm B ซึ่งประเมิน Infix โดยตรง จากการทดสอบ Test Cases ระบบสามารถประมวลผลนิพจน์ที่ถูกต้องและตรวจสอบ Error Cases ได้

เปรียบเทียบ Algorithm

หัวข้อ	Algorithm A	Algorithm B
วิธีการ	Infix $\rightarrow$ Postfix $\rightarrow$ Evaluate	Direct Infix Evaluation
Time Complexity	$O(n)$	$O(n)$
Space Complexity	$O(n)$	$O(n)$
การจัดการขั้นตอน	แยก 2 ขั้นตอน	แยก 2 ขั้นตอน
ความเข้าใจ	เข้าใจง่ายกว่า	Logic ซับซ้อนกว่า
Postfix	มี	ไม่มี

Algorithm ที่กลุ่มเลือก

กลุ่มเลือก Algorithm A เนื่องจากมีโครงสร้างการทำงานที่ชัดเจน โดยแยกขั้นตอนการแปลง Infix เป็น Postfix ออกจากการประเมินผล ทำให้สามารถตรวจสอบลำดับความสำคัญของ Operator และอธิบายการทำงานของ Stack ได้ง่าย นอกจากนี้ยังมี Time Complexity ในระดับ $O(n)$ และเหมาะสำหรับการศึกษาและพัฒนาระบบ Expression Processor

หน้าที่แต่ละคน

นายกษิตศ อุ๋นอำไพ Presentation

นางสาวน้ำผึ้ง เอียดวงศ์ report

นายชลสิทธิ์ จิตรณรงค์ Test / Algorithm

นายณัฐชนันท์ จินดานิล Diagram / Flowchart

นายพฤทธชาติ ดำนิล github / Algorithm

บันทึกการใช้ Generative AI

Group.md
ไฟล์

Work.md
ไฟล์

ไฟล์ Group คือสิ่งที่ต้องทำ ไฟล์ work คือโจทย์ที่ต้องทำ ช่วยกันทำออกมา
ทั้งหมดเลย

Expected result ดูยังไงในโปรแกรม

อันที่ error คือ pass หมดเลยไหม้ย

Class diagram ต้องใส่ uses / implement ตร...

Flowchart a กับ b ต่างกันตรงไหน

case story คืออะไรครับ ของงานนี้

ตัวอย่าง step by step ของโจทย์นี้